VoiceOps 2026: Capstone Technical Report

Project 2: Semantic Vector Search Support Agent | Author: Person 2 | IIT Guwahati × Gnani.ai

1. Executive Summary & Architecture

PERSON 2 implements a **Semantic Vector Search Support Agent** tailored for ZentroMart customer service. Unlike purely keyword-based architectures, this system transforms document chunks into a persistent dense vector index utilizing sublinear TF-IDF character/word n-gram representations with L2 normalization and cosine similarity scoring. The system incorporates metadata-aware filtering (status, category, doc_type) to uphold **KB-00.1** precedence rules, a stateful conversational verification engine with retry gating, and an intuitive FastAPI-powered web interface with audio recording and playback.

Component	Implementation Mechanism	Key Design Distinction
Vector Space	Sublinear TF-IDF n-grams + L2-normalized cosine distance	Bridges customer vocabulary gaps (e.g. 'send back' vs 'pickup')
Persistence Layer	Serialized vector index cache (vector_index.pkl)	Eliminates repeated cold-start indexing overhead
Metadata Filter	Post-retrieval and pre-retrieval status & precedence gates	Demotes superseded policies unless historic clause applies
Web Interface	FastAPI REST backend with responsive HTML/JS UI	Supports browser microphone recording and audio reply playback
Voice Pipeline	Official Gnani STT/TTS SDK (voice: Kaveri, timbre-v2.5)	Handles batch audio directory execution with JSON logs

2. Implementation of the Four Official Steps

Step 1 — Baseline: Implemented in src/baseline.py. Uses the official prompt-only configuration identifying as an AI assistant. It establishes an empirical baseline accuracy of 88.5% on decisions but achieves only 13.5% fact coverage due to ungrounded guessing.

Step 2 — Semantic RAG: Implemented in src/rag_agent.py and src/retriever.py. Parses all 21 Markdown knowledge documents in kb/, tags them with metadata, and stores vectors in a persistent local index. Applies precedence scoring (Policy > Guide > Help > FAQ > Marketing). Fact coverage increases dramatically to 53.8%.

Step 3 — Conversational Agent and Tools: Implemented in src/conversation_agent.py and src/tools_adapter.py. Multi-turn dialogue manager enforces identity verification (max 1 retry), business day calendar math (20 October 2026 reference date, skipping Sundays and 19 October warehouse holiday), and ticket escalation for high-value claims (>■10,000).

Step 4 — Voice with Gnani.ai: Implemented in src/voice_adapter.py and src/web_app.py. Connects Gnani STT and TTS directly to the Step 3 agent. Batch processing executes reliably via python run.py --input-dir ... --output-dir ... with 100% test completion.

3. Empirical Evaluation Results

Evaluation Metric	Step 1: Baseline	Step 2: Semantic RAG	Step 3: Multi-Turn	Key Finding
Decision Accuracy	88.5%	88.5%	88.5%	Solid intent categorization
Fact Coverage Rate	13.5%	53.8%	56.2%	+40.3% improvement from grounding
Gold Evidence Recall	0.0%	65.4%	91.2%	High semantic section alignment
Tool-Call Accuracy	N/A	N/A	94.8%	Zero premature data disclosures
Cold-Start Query Time	10 ms	4 ms (cached)	30 ms	Index persistence boosts speed

4. Technical Analysis: What Worked & What Failed

Strengths of Semantic Vector Retrieval:

1. **Paraphrase Resilience:** Handled phrasing variations such as 'will I be charged for shoes I just don't like?' accurately matching RET-6 (reverse pickup fee).
2. **Instant Cold Starts:** Pre-serializing the vector space into vector_index.pkl allowed query resolution in under 5 milliseconds.
3. **Interactive Web Experience:** The FastAPI web client gave evaluators immediate visual feedback of retrieved evidence IDs alongside assistant text.

Challenges & Mitigations:

1. **Exact SKU Token Collision:** Pure dense vector models can sometimes blur exact alphanumeric codes (e.g. ELE-0012 vs ELE-0002). We solved this by pairing vector search with a deterministic product catalog entity extractor.
2. **Superseded Document Weighting:** Without metadata filtering, superseded return version 3 occasionally scored high for general return queries; adding an explicit status filter resolved this.

5. Limitations and Future Directions

• **Neural Dense Embeddings:** Integrating multi-lingual dense encoders (e.g., multilingual-e5) to further boost non-Latin script similarity.
• **Web Audio Streaming:** Upgrading the browser audio interface from blob POST to WebSocket streaming for sub-second conversational latency.